

Multithreading

Course: Rust Basics

Author: Aleksandr Kugushev

Thread

```
thread::spawn(|| {  
    for _ in 0..5 {  
        println!("Hello");  
    }  
    thread::sleep(Duration::from_millis(1))  
});  
  
for _ in 0..5 {  
    println!("World");  
    thread::sleep(Duration::from_millis(1))  
}
```

World
Hello
Hello
World
Hello
World
Hello
World
World
Hello

Thread builder

```
thread::Builder::new()  
    .name(String::from("My Thread"))  
    .stack_size(32 * 1024)  
    .spawn(|| {  
        println!("Hello World")  
    });
```

Threads are background

```
thread::spawn(|| {  
    for _ in 0..5 {  
        println!("Hello");  
    }  
});
```

```
thread::sleep(Duration::from_millis(1))  
}
```

Hello

JoinHandle

```
let handle: JoinHandle<i32> = thread::spawn(|| {  
    for _ in 0..5 {  
        println!("Hello");  
        thread::sleep(Duration::from_millis(1))  
    }  
    42  
});
```

```
let result: Result<i32, Box<dyn Any + Send + 'static>> = handle.join();  
assert_eq!(result.unwrap(), 42)
```

Handle error

```
let handle: JoinHandle<i32> = thread::spawn(|| {  
    panic!("AAA!")  
});  
  
let result: Result<i32, Box<dyn Any + Send + 'static>> =  
    handle.join();  
  
let error: Box<dyn Any + Send + 'static> = result.unwrap_err();  
let downcast: Box<&str> = error.downcast:::<&str>().unwrap();  
assert_eq!(*downcast.deref(), "AAA!");
```

Closure

```
let int = 5;  
let handle = thread::spawn(|| { <- may outlive borrowed value `int`  
    println!("Int background {int}");  
});  
handle.join().expect("The thread shouldn't panic 0_o");  
println!("Int main {int}");
```

thread::spawn

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
{
    Builder::new().spawn(f).expect("failed to spawn thread")
}
```

Move to spawn

```
let int = 5;  
let handle = thread::spawn(move || {  
    println!("Int background {int}");  
});  
handle.join().expect("The thread shouldn't panic 0_o");  
println!("Int main {int}");
```

Move to spawn ☹️

```
let int = 5;  
let text = String::from("Int");  
let handle = thread::spawn(move || {  
    println!("{text} background {int}");  
});  
handle.join().expect("The thread shouldn't panic 0_o");  
println!("{text} main {int}");  
      ^^^^ value borrowed here after move
```

thread::scope

```
let int = 5;  
let text = String::from("Int");  
thread::scope(|s: &Scope| {  
    // main thread  
    s.spawn(|| {  
        // background thread  
        println!("{text} background {int}");  
    });  
});  
  
// main thread  
println!("{text} main {int}");
```

thread::scope

```
let int = 5;  
let text = String::from("Int");  
  
thread::scope(|s: &Scope| {  
    // main thread  
    s.spawn(|| {  
        // background thread  
        println!("{text} background {int}");  
    });  
    s.spawn(|| {  
        // background thread  
        println!("{text} background {int}");  
    });  
});  
// main thread  
println!("{text} main {int}");
```

Race Condition

```
fn increment(counter: &mut i32){  
    let buffer = *counter + 1;  
    *counter = buffer;  
}
```

Race Condition

Thread 1

// counter = 0

let buffer = *counter + **1**; *// 1*

**counter = buffer; // 1*

Thread 2

// counter = 0

let buffer = *counter + **1**; *// 1*

**counter = buffer; // 1*

Use & + &mut in scope

```
let mut int = 5;
thread::scope(|s: &Scope| {
    s.spawn(|_| { -- first mutable borrow occurs here
        int = int + 1;
        println!("{text} background {int}");
    });
    s.spawn(|_| { <- second mutable borrow occurs here
        int = int + 1;
        println!("{text} background {int}");
    });
});
println!("{text} main {int}");
```

Use & + &mut in scope

```
let mut int = 5;
thread::scope(|s: &Scope| {
    s.spawn(|| {
        int = int + 1;
        println!("{text} background {int}");
    });
    s.spawn(|| {
        println!("{text} background {int}");
    });
});
println!("{text} main {int}");
```

cannot borrow `int` as immutable because it is also borrowed as mutable

Scope::spawn

```
pub fn spawn<F, T>(&'scope self, f: F) ->
  ScopedJoinHandle<'scope, T>
where
  F: FnOnce() -> T + Send + 'scope,
  T: Send + 'scope,
{
  Builder::new().spawn_scoped(self, f).expect("failed to spawn
thread")
}
```

Scope::spawn

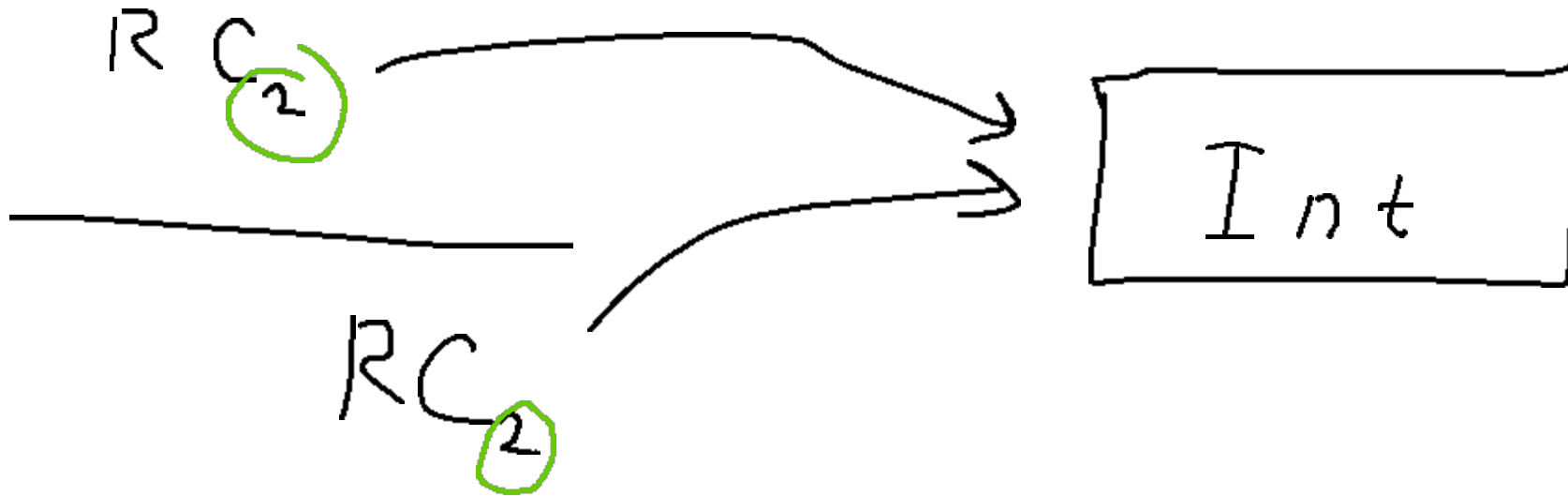
```
pub fn spawn<F, T>(& 'scope self, f: F) ->
  ScopedJoinHandle<'scope, T>
where
  F: FnOnce() -> T + Send + 'scope,
  T: Send + 'scope,
{
  Builder::new().spawn_scoped(self, f).expect("failed to spawn
thread")
}
```

What about Rc?

```
let int = 5;
let text: Rc<String> = Rc::new(String::from("Int"));
let clone = text.clone();
let handle = thread::spawn(move || {
    let txt: &String = clone.deref();
    println!("{txt} background {int}");
});
handle.join().expect("The thread shouldn't panic 0_o");
println!("{text} main {int}");
```

`Rc<String>` cannot be sent between threads safely

Why Rc is not thread safe?



How the compiler found out? 0_o

```
let int = 5;
let text: Rc<String> = Rc::new(String::from("Int"));
let clone = text.clone();
let handle = thread::spawn(move || {
    let txt: &String = clone.deref();
    println!("{txt} background {int}");
});
handle.join().expect("The thread shouldn't panic 0_o");
println!("{text} main {int}");
```

How the compiler found out? 0_o

```
impl<T: ?Sized> !Send for Rc<T> {}
```

```
impl<T: ?Sized> !Sync for Rc<T> {}
```

Send/Sync

```
pub unsafe auto trait Send {  
    // empty.  
}  
  
pub unsafe auto trait Sync {  
    // Empty  
}
```

Auto trait

Auto traits, like `Send` or `Sync` in the standard library, are marker traits that are automatically implemented for every type, unless the type, or a type it contains, has explicitly opted out via a negative impl.

```
impl !Trait for Type {}
```

negative trait bounds are not yet fully implemented;

<https://doc.rust-lang.org/beta/unstable-book/language-features/auto-traits.html>

Send/Sync

Send

- Can be moved

Sync

- Can be referenced

Send/Sync

Send

```
let int = 5;  
thread::spawn(move || {  
    println!("background {int}");  
});  
println!("main {int}");
```

Sync

```
let int = 5;  
thread::scope(|s: &Scope| {  
    s.spawn(|| {  
        println!("background  
{int}");  
    });  
});  
println!("main {int}");
```

Send/Sync

Send + Sync	String, i32, anything except below
Send + !Sync	Cell, RefCell, UnsafeCell, etc.
!Send + !Sync	Rc, raw pointers

Rust as awesome for multithreading because

- `&/&mut` for most cases
- `Send/Sync` for Interior Mutability/Unsafe

Arc

```
let arc1: Arc<String> = Arc::new(String::from("Shared ownership"));
let arc2: Arc<String> = arc1.clone();
let handle = thread::Builder::new().name(String::from("background"))
    .spawn(|| {
        let moved_arc2: Arc<String> = arc2;
        println!("{}", from {},
            moved_arc2,
            thread::current().name().unwrap());
    });
println!("{}", from {},
    arc1,
    thread::current().name().unwrap());
handle.unwrap().join().expect("The thread shouldn't panic 0_o");
```

> Shared ownership
from main

> Shared ownership
from background

Arc<T> where T: Send & Sync

```
let arc1: Arc<RefCell<String>> = Arc::new(
    RefCell::new(
        String::from("Shared ownership")));
let arc2: Arc<RefCell<String>> = arc1.clone();
let handle = thread::Builder::new().name(String::from("background"))
    .spawn(|| {
        let moved_arc2: Arc<RefCell<String>> = arc2;
        let mut string = moved_arc2.borrow_mut();
        string.push_str(" and updated")
    });
println!("{}",
    arc1.borrow(),
    thread::current().name().unwrap());
handle.unwrap().join().expect("The thread shouldn't panic 0_o");
```

<- `RefCell<String>` cannot be shared between threads safely

std::sync::atomic

AtomicBool

AtomicI8

AtomicI16

AtomicI32

AtomicI64

AtomicIsize

AtomicPtr

AtomicU8

AtomicU16

AtomicU32

AtomicU64

AtomicUsize

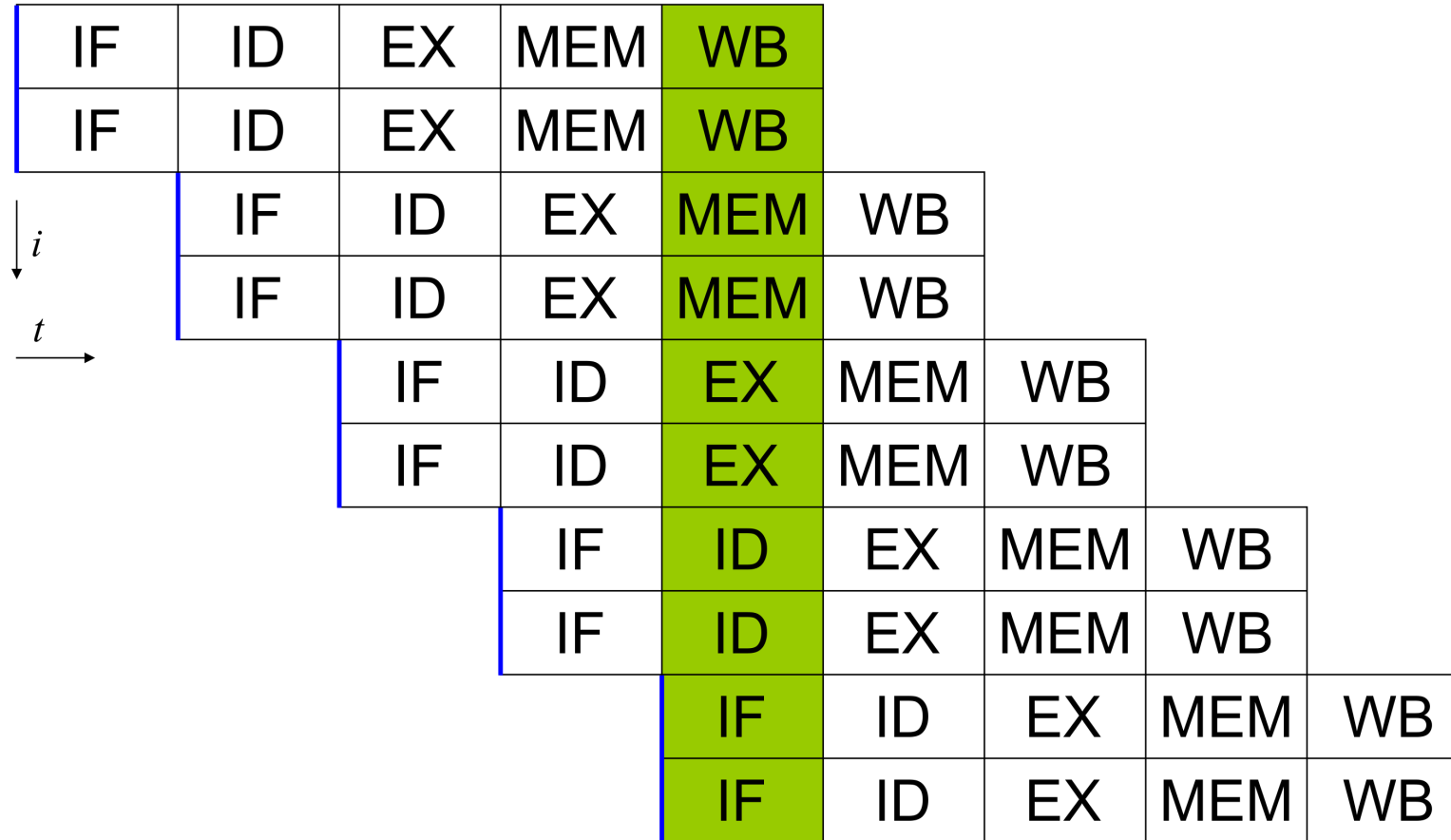
- Interior Mutability
- Prevents the compiler and CPU from reordering
- Atomic operations

static *MY_COUNTER*: AtomicI32

```
static MY_COUNTER: AtomicI32 = AtomicI32::new(0);
```

```
let handle = thread::spawn(|| {  
    for _ in 0..100000 {  
        MY_COUNTER.fetch_add(1, Ordering::SeqCst); // increment  
    }  
});  
for _ in 0..100000 {  
    MY_COUNTER.fetch_add(1, Ordering::SeqCst); // increment  
}  
handle.join().expect("The thread shouldn't panic 0_o");  
println!("Counter={}", MY_COUNTER.load(Ordering::SeqCst)); // 200000
```

Reordering



std::sync::atomic::fence

```
fence(Ordering::Acquire),  
let value: i32 = MY_COUNTER.load();
```



// do something

```
MY_COUNTER.store(value);
```

```
fence(Ordering::Release);
```



Ordering

- Relaxed
- Acquire – choose on read
 - Any memory operation written after the Acquire stays after it
- Release – choose on write
 - Any memory operation written before the Release stays before it
- AcqRel – choose for compare_exchange
- SeqCst – choose if you're not sure

Atomic operations

```
let handle = thread::spawn(|| {  
    for _ in 0..1000 {  
        timely_increment(&MY_COUNTER);  
    }  
});  
for _ in 0..1000 {  
    timely_increment(&MY_COUNTER);  
}  
handle.join().expect("The thread shouldn't panic 0_o"); // fence inside  
println!("Counter={}", MY_COUNTER.load(Ordering::Relaxed)); //  
2000
```

compare_exchange

```
fn timely_increment(value: &AtomicI32) {  
    loop {  
        let mut current = value.load(Ordering::Acquire);  
        let new = current + 1;  
        thread::sleep(Duration::from_millis(1));  
        let result = value.compare_exchange(current, new,  
                                           Ordering::AcqRel,  
                                           Ordering::Acquire);  
  
        if result.is_ok() {  
            return;  
        }  
    }  
}
```

Send/Sync and Pointers

```
struct NoSendStruct{  
    cell: *const i32  
}
```

...

```
let str = NoSendStruct {  
    cell: &42  
};
```

```
thread::spawn(move || {  
    let moved = str;  
    println!("{}", unsafe { *moved.cell });  
});
```

error[E0277]: `\*const i32` cannot be sent between threads safely

Send/Sync and Pointers

```
struct NoSendStruct{  
    cell: *const i32  
}  
  
unsafe impl Send for NoSendStruct {}  
  
...  
let str = NoSendStruct {  
    cell: &42  
};  
  
thread::spawn(move || {  
    let moved = str;  
    println!("{}", unsafe { *moved.cell });  
});
```

RAll based locks

- Mutex
- RwLock
- ...

Mutex

```
let mutex = Mutex::new(String::from("Hello"));

thread::scope(|s| {
    s.spawn(|_| {
        let mut guard: MutexGuard<String> = mutex.lock().unwrap();
        guard.push_str(" World");
    });
});

println!("{}", mutex.lock().unwrap())
> Hello World
```

Poisoning

```
let mutex = Mutex::new(String::from("Hello"));
thread::scope(|s| {
    let handle = s.spawn(|_| {
        let mut guard: MutexGuard<String> = mutex.lock().unwrap();
        guard.push_str(" World");
        panic!("Something went wrong!")
    });
    assert!(handle.join().is_err())
});
assert_eq!(mutex.is_poisoned(), true);
let result: LockResult<MutexGuard<String>> = mutex.lock();
assert!(result.is_err());
let guard: MutexGuard<String> = result.unwrap_err().into_inner();
println!("{}", guard)
```

Arc<Mutex<T>>

```
let arc1: Arc<Mutex<String>> = Arc::new(
    Mutex::new(
        String::from("Shared ownership")));
let arc2: Arc<Mutex<String>> = arc1.clone();
let handle = thread::Builder::new().name(String::from("background"))
    .spawn(|| {
        let moved_arc2: Arc<Mutex<String>> = arc2;
        let mut guard: MutexGuard<String> = moved_arc2.lock().unwrap();
        guard.push_str(" and updated")
    });
while !handle.as_ref().unwrap().is_finished() {
    let guard = arc1.lock().unwrap();
    println!("{}", guard, thread::current().name().unwrap());
}
```

- [illegible]

RwLock<T>

```
let arc1: Arc<RwLock<String>> = Arc::new(RwLock::new(String::from("0"))); > 0
let arc2: Arc<RwLock<String>> = arc1.clone(); > 0
let handle = thread::Builder::new().name(String::from("background")) > 0
    .spawn(|| { > 0
        let moved_arc2: Arc<RwLock<String>> = arc2; > 26
        for i in 1..100 { > 26
            let mut guard: RwLockWriteGuard<String> = moved_arc2.write().unwrap(); > 26
            let len = guard.len(); > 77
            guard.replace_range(0..len, &i.to_string()) > 77
        } > 77
    }); > 77
while !handle.as_ref().unwrap().is_finished() { > 77
    let guard: RwLockReadGuard<String> = arc1.read().unwrap(); > 77
    println!("{guard}"); > 99
}
```

RwLock<T>

```
let arc1: Arc<RwLock<String>> = Arc::new(RwLock::new(String::from("0"))); > 0
let arc2: Arc<RwLock<String>> = arc1.clone(); > 0
let handle = thread::Builder::new().name(String::from("background")) > 0
    .spawn(|| { > 0
        let moved_arc2: Arc<RwLock<String>> = arc2; > 0
        let mut guard: RwLockWriteGuard<String> = moved_arc2.write().unwrap(); > 0
        for i in 1..100 { > 0
            let len = guard.len(); > 0
            guard.replace_range(0..len, &i.to_string()) > 0
        } > 0
    }); > 0
while !handle.as_ref().unwrap().is_finished() { > 0
    let guard: RwLockReadGuard<String> = arc1.read().unwrap(); > 0
    println!("{}", guard); > 0
} PAUSE!
> 99
```

Channels

```
let (tx, rx): (Sender<String>, Receiver<String>) = mpsc::channel();
let handle = thread::spawn(move || {
    for i in 1..100 {
        tx.send(i.to_string());
    }
});
while !handle.is_finished() {
    let next: Result<String, RecvError> = rx.recv();
    println!("{}", next.unwrap());
}
```

> 0
> 1
> 2
> 3
> 4
> 5
> 6
> 7
> 8
> 9
> 10
> 12

Channels and ownership

```
pub fn send(&self, t: T) ->  
Result<(), SendError<T>> {  
    self.inner.send(t)  
}
```

```
pub fn recv(&self) -> Result<T,  
RecvError> {  
    self.inner.recv()  
}
```

Receiver<T> is iterator

```
let (tx, rx): (Sender<String>, Receiver<String>) = mpsc::channel();
let handle = thread::spawn(move || {
    for i in 1..100 {
        tx.send(i.to_string());
    }
});

for val in rx {
    println!("{}", val);
}
```

Multiple Producer Single Consumer

```
let (tx, rx): (Sender<String>, Receiver<String>) = mpsc::channel();
let tx_clone = tx.clone();
thread::spawn(move || {
    for i in 1..100 {
        tx.send(i.to_string());
    }
});
thread::spawn(move || {
    for i in 100..10000 {
        tx_clone.send(i.to_string());
    }
});
for val in rx {
    println!("{val}");
}
```

Homework

- Rustlings:
 - Threads
- Courseware
 - Multithreading search - should use all cores